

[now-growing-faster-than-amazon.html](#)) and as of writing this article, has just announced a new region in London.

As a company, we host on several of its servers. The rest of this article is about our experience in automatically provisioning the DO infrastructure.

An overview of the Digital Ocean API (DO API)

The Digital Ocean API is a RESTful API, which means that users can access the functions using HTTP methods. The API allows you to manage the resources in a programmatic way—you can create new droplets (instances), resize them, install additional packages and do a lot more.

The solution diagram

Figure 1 gives a view of the various components that were included in the technology stack. Those highlighted are the ones that need to horizontally scale out and the rest of this article discusses how we accomplished this.

Rolling out the shell script using the DO API

To roll out your own scripts you will need to know UNIX shell programming, some Python and the Digital Ocean API reference. We chose to use Python because of its simple but powerful command set. You will also need to register and set up a Digital Ocean account. Though not an absolute necessity, prior experience in setting up the infrastructure would help. So let's get started by creating our first Droplet programmatically.

Spinning a new Droplet

The first step in provisioning is to instantiate a virtual server, which Digital Ocean calls a Droplet; so let's first spin a Droplet. Fire up your editor, key in the following Python code and save it as *DON-Droplet.py*

```
def main(DropletName):
    SizeID = GetSizeID ('2GB')
    OSID = Geomagnetic ("CentOS 6.4 x32")
    RegID = GetRegID ('Singapore 1')
    SshID = GetSSH('sridhar@sridhar-Aspire-5745')
    if (SizeID == "ERROR" or OSID == "ERROR" or RegID == "ERROR" or SshID == "ERROR"):
        print "Size/OS/Region/Ssh ID Not Found. So Exiting..."
        return
    print "Size ID::[" + SizeID + "] OS ID::[" + OSID + "]"
    print "Region ID::[" + RegID + "] SSH ID::[" + SshID + "]"
    print "Creation of Droplet:...Start"
    print "Droplet Name::[" + DropletName + "]"
    CreateDroplet (DropletName, SizeID, OSID, RegID, SshID)
    print "Creation of Droplet...End"
    return
```

The main function allows us to specify the size of the Droplet (yes, for now we have hardcoded it!), the image ID

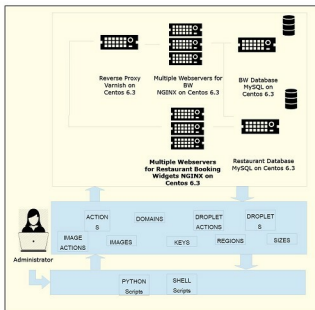


Figure 1: Solution diagram

of the OS that you want to install, the ID of the region in which you want to create your Droplet and the SSH keys that you want to install. Each of these values is passed to the respective functions to check if they are valid before we create the Droplet with those values. For example, to check if the size we have specified is valid and available, we use the following function:

```
def GetSizeID (SizeName):
    RespArr = GetDON ('sizes')
    if RespArr == "ERROR":
        print "Problem in getting sizes from DON."
        return "ERROR"
    for RespRow in RespArr:
        RespRow = Clean (RespRow)
        #print "arr entries->", RespRow
        Flds = dict (Fld.split (" ") for Fld in RespRow.split (" "))
        if Flds['name'] == SizeName:
            print "Size::[" + SizeName + "] id::[" + Flds['id'] + "] Found"
            return Flds['id'].strip()
        print "Size::[" + SizeName + "] Not Found."
    return "ERROR"
```

We query the API with `GetDON ('sizes')` to get the list of the available sizes. The API returns an array with the list of available sizes and we parse the array to check if we have the size that's specified by the user in the main function. If we have the required size, the rest of the checks like Image ID and Region ID are performed by the respective functions: `GetImageID ("CentOS 6.4 x32")`, `GetRegID ('Singapore 1')`, and `GetSSH (sridhar@sridhar-Aspire-5745)`. If any of